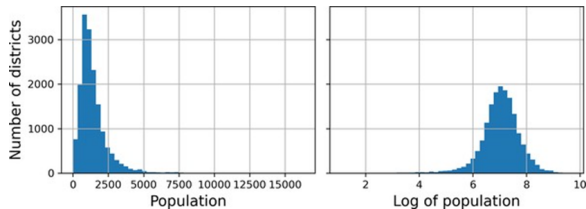


Biến đổi Đặc trưng để gần với phân phối Gaussian

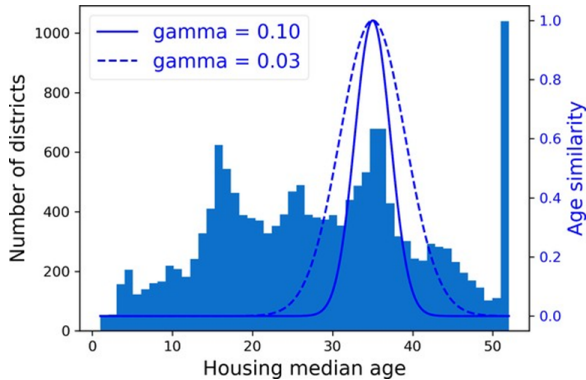
- Nhiều biểu đồ tần suất bị "nặng đuôi" (tail-heavy).
- Sử dụng các phép tính logarit (Log) hoặc logarit căn bản để biến đổi dữ liệu về dạng giống cái chuông (bell-shaped Gaussian).



Hình 2-17. Biến đổi đặc trưng nặng đuôi

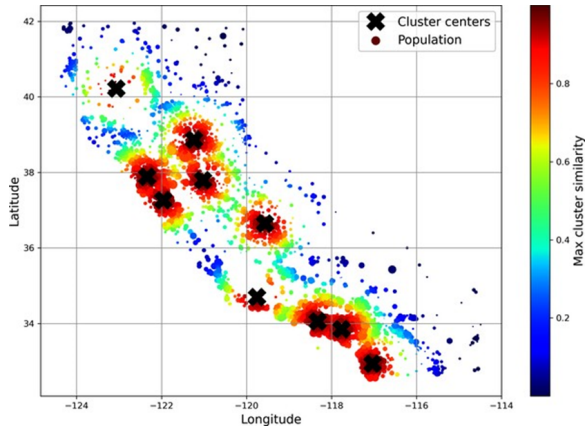
Biến đổi Đặc trưng bằng RBF

- Đối với phân phối có nhiều đỉnh (multimodal).
- Dùng **Radial Basis Function (RBF)** - Hàm cơ sở xuyên tâm. Đo khoảng cách (sự tương đồng) giữa một giá trị và một "đỉnh" (landmark) cố định.



Hình 2-18. Đặc trưng RBF (Đo khoảng cách tới 35)

Đặc trưng từ phân cụm bằng K-Means



Hình 2-19. Đặc trưng RBF với tâm cụm từ K-Means

Bộ biến đổi tùy chỉnh (Custom Transformers)

- Mặc dù Scikit-Learn cung cấp nhiều bộ biến đổi, ta vẫn cần tự viết mã để thực hiện logic riêng (ví dụ: tính số phòng trên mỗi hộ).
- Tạo các lớp (Class) tùy chỉnh tuân theo API của Scikit-Learn (kế thừa từ `BaseEstimator` và `TransformerMixin`).

Các pipeline biến đổi (Transformation Pipelines)

- Quá trình biến đổi dữ liệu yêu cầu thực hiện nhiều bước theo đúng thứ tự.
- Pipeline trong Scikit-Learn giúp liên kết các bước xử lý dữ liệu lại với nhau thành một luồng duy nhất.
- ColumnTransformer: Áp dụng các luồng xử lý khác nhau cho các loại cột khác nhau (VD: Cột số chạy qua StandardScaler, cột phân loại chạy qua OneHotEncoder).

3. Chọn, Huấn luyện, Tinh chỉnh & Triển khai

Huấn luyện trên Tập huấn luyện

- Bắt đầu với một mô hình đơn giản như **Hồi quy tuyến tính (Linear Regression)**.
- Nếu RMSE quá cao (ví dụ sai số 68,000 USD), nghĩa là mô hình bị **Underfitting** (chưa khớp).
- Thử nghiệm với một mô hình phức tạp hơn: **Cây quyết định (Decision Tree Regressor)**. Mô hình mạnh mẽ, có thể tìm ra các mối quan hệ phi tuyến tính phức tạp.

Đánh giá tốt hơn bằng kiểm định chéo (Cross-Validation)

- Sử dụng **K-fold Cross-Validation**.
- Chia ngẫu nhiên tập huấn luyện thành K tập con (fold) khác biệt.
- Huấn luyện và đánh giá mô hình K lần, mỗi lần chọn một fold khác nhau để đánh giá và K-1 fold còn lại để huấn luyện.
- Giúp biết chính xác mô hình tổng quát hóa tốt như thế nào mà không cần chạm vào tập kiểm thử thực (Test set).

Thử nghiệm với Rừng ngẫu nhiên (Random Forest)

- Xây dựng nhiều Cây quyết định trên các tập hợp con ngẫu nhiên của các đặc trưng, sau đó lấy trung bình dự đoán của chúng.
- Đây là một mô hình **Học tập hợp (Ensemble Learning)**.
- Thường cho kết quả vượt trội và là một trong những mô hình máy học mạnh nhất.

Tinh chỉnh mô hình

Tìm kiếm theo lưới (Grid Search)

- Sử dụng GridSearchCV của Scikit-Learn để tự động thử nghiệm tất cả các tổ hợp có thể có của các siêu tham số.
- Rất hữu ích khi khám phá một vài kết hợp, nhưng cực kỳ tốn thời gian tính toán nếu không gian tìm kiếm lớn.

Tìm kiếm ngẫu nhiên (Random Search)

- Dùng RandomizedSearchCV: thay vì thử nghiệm tất cả các kết hợp (như Grid Search), nó sẽ đánh giá một số lượng ngẫu nhiên nhất định.
- Cho phép điều chỉnh siêu tham số với hiệu suất tính toán tốt hơn nếu không gian tìm kiếm quá lớn (nhiều siêu tham số).

Các phương pháp tập hợp (Ensemble Methods)

- Một cách khác để tinh chỉnh hệ thống là cố gắng kết hợp các mô hình hoạt động tốt nhất.
- Một nhóm (ensemble) thường sẽ hoạt động tốt hơn so với mô hình cá nhân tốt nhất.
(Giống như quyết định của một hội đồng thường tốt hơn một cá nhân).

Phân tích các mô hình tốt nhất và lỗi của chúng

- Có thể đạt được những cái nhìn sâu sắc bằng cách kiểm tra các mô hình tốt nhất.
- Mô hình RandomForest có thể cho biết mức độ quan trọng tương đối của từng đặc trưng.
- Dựa vào thông tin này, bạn có thể cân nhắc loại bỏ một số đặc trưng ít hữu ích nhất.

Đánh giá hệ thống trên tập kiểm thử

- Khi đã hài lòng với mô hình cuối cùng, đây là lúc kiểm tra hiệu suất trên **Test Set**.
- Chạy tập kiểm thử qua `full_pipeline` để chuyển đổi dữ liệu. **Lưu ý: Chỉ gọi `transform()`, không gọi `fit()`.**
- Hiệu suất thường sẽ hơi tệ hơn so với trên tập xác thực (do mô hình đã được tinh chỉnh cho tập xác thực). Đừng cố tình điều chỉnh siêu tham số nữa!